# 5
# Creating a Panoramic Image

In this chapter, we are going to learn how to stitch multiple images of the same scene together to create a panoramic image.

By the end of this chapter, you will know:

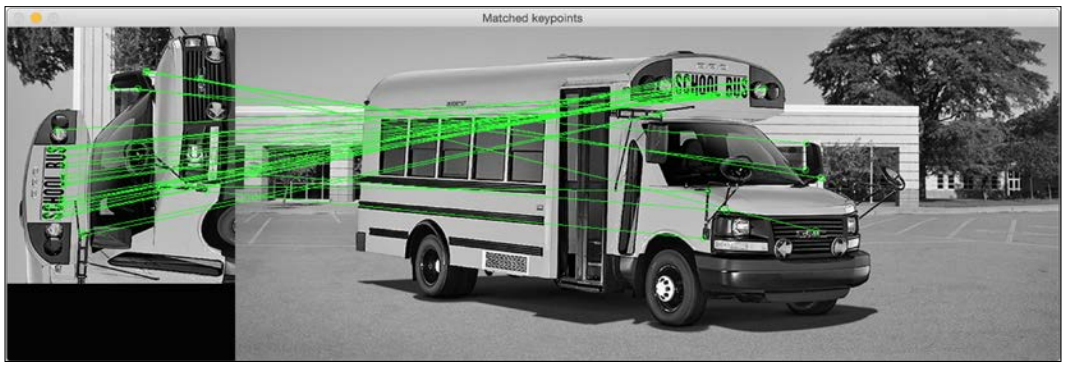
- How to match keypoint descriptors between multiple images
- How to find overlapping regions between images
- How to warp images based on the matching keypoints
- How to stitch multiple images to create a panoramic image

## Matching keypoint descriptors

In the last chapter, we learned how to extract keypoints using various methods. The reason that we extract keypoints is because we can use them for image matching. Let's consider the following image:

As you can see, it's the picture of a school bus. Now, let's take a look at the following image:


The preceding image is a part of the school bus image and it's been rotated anticlockwise by 90 degrees. We could easily recognize this because our brain is invariant to scaling and rotation. Our goal here is to find the matching points between these two images. If you do that, it would look something like this:



Following is the code to do this:

```
import sys

import cv2
import numpy as np

def draw_matches(img1, keypoints1, img2, keypoints2, matches):
    rows1, cols1 = img1.shape[:2]
    rows2, cols2 = img2.shape[:2]

    # Create a new output image that concatenates the two images
together
    output_img = np.zeros((max([rows1,rows2]), cols1+cols2, 3),
dtype='uint8')
    output_img[:rows1, :cols1, :] = np.dstack([img1, img1, img1])
    output_img[:rows2, cols1:cols1+cols2, :] = np.dstack([img2, img2,
img2])

    # Draw connecting lines between matching keypoints
    for match in matches:
        # Get the matching keypoints for each of the images
        img1_idx = match.queryIdx
        img2_idx = match.trainIdx

        (x1, y1) = keypoints1[img1_idx].pt
        (x2, y2) = keypoints2[img2_idx].pt

        # Draw a small circle at both co-ordinates and then draw a
line
        radius = 4
        colour = (0,255,0)   # green
        thickness = 1
        cv2.circle(output_img, (int(x1),int(y1)), radius, colour,
thickness)
        cv2.circle(output_img, (int(x2)+cols1,int(y2)), radius,
colour, thickness)
        cv2.line(output_img, (int(x1),int(y1)),
(int(x2)+cols1,int(y2)), colour, thickness)

    return output_img

if __name__=='__main__':
    img1 = cv2.imread(sys.argv[1], 0)   # query image (rotated
subregion)
```

```
    img2 = cv2.imread(sys.argv[2], 0)   # train image (full image)

    # Initialize ORB detector
    orb = cv2.ORB()

    # Extract keypoints and descriptors
    keypoints1, descriptors1 = orb.detectAndCompute(img1, None)
    keypoints2, descriptors2 = orb.detectAndCompute(img2, None)

    # Create Brute Force matcher object
    bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

    # Match descriptors
    matches = bf.match(descriptors1, descriptors2)

    # Sort them in the order of their distance
    matches = sorted(matches, key = lambda x:x.distance)

    # Draw first 'n' matches
    img3 = draw_matches(img1, keypoints1, img2, keypoints2,
matches[:30])

    cv2.imshow('Matched keypoints', img3)
    cv2.waitKey()
```

# How did we match the keypoints?

In the preceding code, we used the ORB detector to extract the keypoints. Once we extracted the keypoints, we used the Brute Force matcher to match the descriptors. Brute Force matching is pretty straightforward! For every descriptor in the first image, we match it with every descriptor in the second image and take the closest one. To compute the closest descriptor, we use the Hamming distance as the metric, as shown in the following line:

```
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
```

You can read more about the Hamming distance at `https://en.wikipedia.org/wiki/Hamming_distance`. The second argument in the preceding line is a Boolean variable. If this is true, then the matcher returns only those keypoints that are closest to each other in both directions. This means that if we get (i, j) as a match, then we can be sure that the i-th descriptor in the first image has the j-th descriptor in the second image as its closest match and vice versa. This increases the consistency and robustness of descriptor matching.

# Understanding the matcher object

Let's consider the following line again:

```
matches = bf.match(descriptors1, descriptors2)
```

Here, the variable matches is a list of DMatch objects. You can read more about it in the OpenCV documentation. We just need to quickly understand what it means because it will become increasingly relevant in the upcoming chapters. If we are iterating over this list of DMatch objects, then each item will have the following attributes:

- **item.distance**: This attribute gives us the distance between the descriptors. A lower distance indicates a better match.
- **item.trainIdx**: This attribute gives us the index of the descriptor in the list of train descriptors (in our case, it's the list of descriptors in the full image).
- **item.queryIdx**: This attribute gives us the index of the descriptor in the list of query descriptors (in our case, it's the list of descriptors in the rotated subimage).
- **item.imgIdx**: This attribute gives us the index of the train image.

# Drawing the matching keypoints

Now that we know how to access different attributes of the matcher object, let's see how we can use them to draw the matching keypoints. OpenCV 3.0 provides a direct function to draw the matching keypoints, but we will not be using that. It's better to take a peek inside to see what's happening underneath.

We need to create a big output image that can fit both the images side by side. So, we do that in the following line:

```
output_img = np.zeros((max([rows1,rows2]), cols1+cols2, 3),
dtype='uint8')
```

As we can see here, the number of rows is set to the bigger of the two values and the number of columns is simply the sum of both the values. For each item in the list of matches, we extract the locations of the matching keypoints, as we can see in the following lines:

```
(x1, y1) = keypoints1[img1_idx].pt
(x2, y2) = keypoints2[img2_idx].pt
```

Once we do that, we just draw circles on those points to indicate their locations and then draw a line connecting the two points.

# Creating the panoramic image

Now that we know how to match keypoints, let's go ahead and see how we can stitch multiple images together. Consider the following image:


Let's say we want to stitch the following image with the preceding image:

If we stitch these images, it will look something like the following one:


Now let's say we captured another part of this house, as seen in the following image:

If we stitch the preceding image with the stitched image we saw earlier, it will look something like this:


We can keep stitching images together to create a nice panoramic image. Let's take a look at the code:

```
import sys
import argparse

import cv2
import numpy as np

def argument_parser():
    parser = argparse.ArgumentParser(description='Stitch two
images together')
    parser.add_argument("--query-image", dest="query_image",
required=True,
            help="First image that needs to be stitched")
    parser.add_argument("--train-image", dest="train_image",
required=True,
            help="Second image that needs to be stitched")
    parser.add_argument("--min-match-count",
dest="min_match_count", type=int,
            required=False, default=10, help="Minimum number of
matches required")
    return parser
```

```
# Warp img2 to img1 using the homography matrix H
def warpImages(img1, img2, H):
    rows1, cols1 = img1.shape[:2]
    rows2, cols2 = img2.shape[:2]

    list_of_points_1 = np.float32([[0,0], [0,rows1],
[cols1,rows1], [cols1,0]]).reshape(-1,1,2)
    temp_points = np.float32([[0,0], [0,rows2], [cols2,rows2],
[cols2,0]]).reshape(-1,1,2)
    list_of_points_2 = cv2.perspectiveTransform(temp_points, H)
    list_of_points = np.concatenate((list_of_points_1,
list_of_points_2), axis=0)

    [x_min, y_min] = np.int32(list_of_points.min(axis=0).ravel() -
0.5)
    [x_max, y_max] = np.int32(list_of_points.max(axis=0).ravel() +
0.5)
    translation_dist = [-x_min,-y_min]
    H_translation = np.array([[1, 0, translation_dist[0]], [0, 1,
translation_dist[1]], [0,0,1]])

    output_img = cv2.warpPerspective(img2, H_translation.dot(H),
(x_max-x_min, y_max-y_min))
    output_img[translation_dist[1]:rows1+translation_dist[1],
translation_dist[0]:cols1+translation_dist[0]] = img1

    return output_img

if __name__=='__main__':
    args = argument_parser().parse_args()
    img1 = cv2.imread(args.query_image, 0)
    img2 = cv2.imread(args.train_image, 0)
    min_match_count = args.min_match_count

    cv2.imshow('Query image', img1)
    cv2.imshow('Train image', img2)

    # Initialize the SIFT detector
    sift = cv2.SIFT()

    # Extract the keypoints and descriptors
    keypoints1, descriptors1 = sift.detectAndCompute(img1, None)
    keypoints2, descriptors2 = sift.detectAndCompute(img2, None)
```

```
    # Initialize parameters for Flann based matcher
    FLANN_INDEX_KDTREE = 0
    index_params = dict(algorithm = FLANN_INDEX_KDTREE, trees = 5)
    search_params = dict(checks = 50)

    # Initialize the Flann based matcher object
    flann = cv2.FlannBasedMatcher(index_params, search_params)

    # Compute the matches
    matches = flann.knnMatch(descriptors1, descriptors2, k=2)

    # Store all the good matches as per Lowe's ratio test
    good_matches = []
    for m1,m2 in matches:
        if m1.distance < 0.7*m2.distance:
            good_matches.append(m1)

    if len(good_matches) > min_match_count:
        src_pts = np.float32([ keypoints1[good_match.queryIdx].pt
for good_match in good_matches ]).reshape(-1,1,2)
        dst_pts = np.float32([ keypoints2[good_match.trainIdx].pt
for good_match in good_matches ]).reshape(-1,1,2)

        M, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC,
5.0)
        result = warpImages(img2, img1, M)
        cv2.imshow('Stitched output', result)

        cv2.waitKey()

    else:
        print "We don't have enough number of matches between the
two images."
        print "Found only %d matches. We need at least %d
matches." % (len(good_matches), min_match_count)
```

# Finding the overlapping regions

The goal here is to find the matching keypoints so that we can stitch the images together. So, the first step is to get these matching keypoints. As discussed in the previous section, we use a keypoint detector to extract the keypoints, and then use a Flann based matcher to match the keypoints.

You can learn more about Flann at `http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.192.5378&rep=rep1&type=pdf`.

The Flann based matcher is faster than Brute Force matching because it doesn't compare each point with every single point on the other list. It only considers the neighborhood of the current point to get the matching keypoint, thereby making it more efficient.

Once we get a list of matching keypoints, we use Lowe's ratio test to keep only the strong matches. David Lowe proposed this ratio test in order to increase the robustness of SIFT.

You can read more about this at `http://www.cs.ubc.ca/~lowe/papers/ijcv04.pdf`.

Basically, when we match the keypoints, we reject the matches in which the ratio of the distances to the nearest neighbor and the second nearest neighbor is greater than a certain threshold. This helps us in discarding the points that are not distinct enough. So, we use that concept here to keep only the good matches and discard the rest. If we don't have sufficient matches, we don't proceed further. In our case, the default value is 10. You can play around with this input parameter to see how it affects the output.

If we have a sufficient number of matches, then we extract the list of keypoints in both the images and extract the homography matrix. If you remember, we have already discussed homography in the first chapter. So if you have forgotten about it, you may want to take a quick look. We basically take a bunch of points from both the images and extract the transformation matrix.

# Stitching the images

Now that we have the transformation, we can go ahead and stitch the images. We will use the transformation matrix to transform the second list of points. We keep the first image as the frame of reference and create an output image that's big enough to hold both the images. We need to extract information about the transformation of the second image. We need to move it into this frame of reference to make sure it aligns with the first image. So, we have to extract the translation information and then warp it. We then add the first image into this and construct the final output. It is worth mentioning that this works for images with different aspect ratios as well. So, if you get a chance, try it out and see what the output looks like.

# What if the images are at an angle to each other?

Until now, we were looking at images that were on the same plane. Stitching those images was straightforward and we didn't have to deal with any artifacts. In real life, you cannot capture multiple images on exactly the same plane. When you are capturing multiple images of the same scene, you are bound to tilt your camera and change the plane. So the question is, will our algorithm work in that scenario? As it turns out, it can handle those cases as well.

Let's consider the following image:


Now, let's consider another image of the same scene. It's at an angle with respect to the first image, and it's partially overlapping as well:


Let's consider the first image as our reference. If we stitch these images using our algorithm, it will look something like this:

If we keep the second image as our reference, it will look something like this:


# Why does it look stretched?

If you observe, a portion of the output image corresponding to the query image looks stretched. It's because the query image is transformed and adjusted to fit into our frame of reference. The reason it looks stretched is because of the following lines in our code:

```
M, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
result = warpImages(img2, img1, M)
```

Since the images are at an angle with respect to each other, the query image will have to undergo a perspective transformation in order to fit into the frame of reference. So, we transform the query image first, and then stitch it into our main image to form the panoramic image.

# Summary

In this chapter, we learned how to match keypoints among multiple images. We discussed how to stitch multiple images together to create a panoramic image. We learned how to deal with images that are not on the same plane.

In the next chapter, we are going to discuss how to do content-aware image resizing by detecting "interesting" regions in the image.

# 6
# Seam Carving

In this chapter, we are going to learn about content-aware image resizing, which is also known as seam carving. We will discuss how to detect "interesting" parts in an image and how to use that information to resize a given image without deteriorating those interesting parts.

By the end of this chapter, you will know:

- What is content awareness
- How to quantify "interesting" parts in an image
- How to use dynamic programming for image content analysis
- How to increase and decrease the width of an image without deteriorating the interesting regions while keeping the height constant
- How to make an object disappear from an image

# Why do we care about seam carving?

Before we start our discussion about seam carving, we need to understand why it is needed in the first place. Why should we care about the image content? Why can't we just resize the given image and move on with our lives? Well, to answer that question, let's consider the following image:

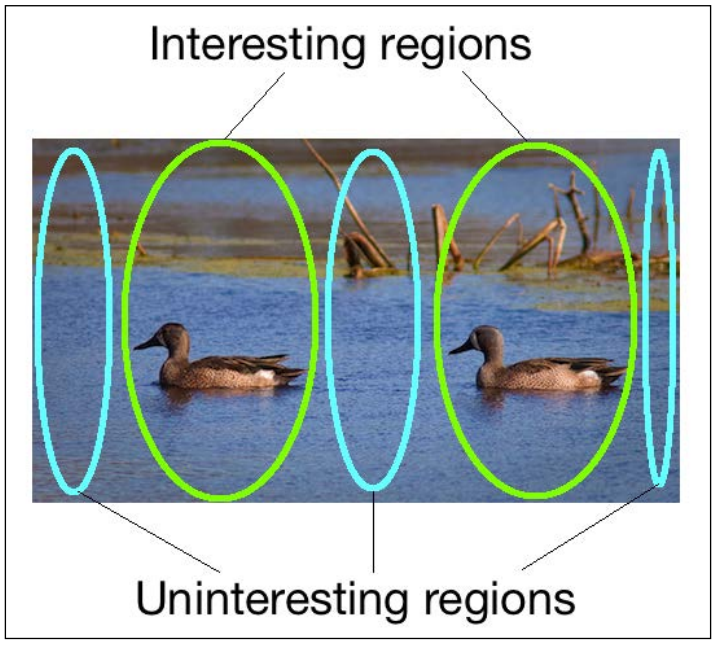
Now, let's say we want to reduce the width of this image while keeping the height constant. If you do that, it will look something like this:

As you can see, the ducks in the image look skewed, and there's degradation in the overall quality of the image. Intuitively speaking, we can say that the ducks are the "interesting" parts in the image. So when we resize it, we want the ducks to be intact. This is where seam carving comes into the picture. Using seam carving, we can detect these interesting regions and make sure they don't get degraded.

# How does it work?

We have been talking about image resizing and how we should consider the image's content when we resize it. So, why on earth is it called seam carving? It should just be called content-aware image resizing, right? Well, there are many different terms that are used to describe this process, such as image retargeting, liquid scaling, seam carving, and so on. The reason it's called seam carving is because of the way we resize the image. The algorithm was proposed by Shai Avidan and Ariel Shamir. You can refer to the original paper at `http://dl.acm.org/citation.cfm?id=1276390`.

We know that the goal is to resize the given image and keep the interesting content intact. So, we do that by finding the paths of least importance in that image. These paths are called seams. Once we find these seams, we remove them from the image to obtain a rescaled image. This process of removing, or "carving", will eventually result in a resized image. This is the reason we call it "seam carving". Consider the image that follows:



In the preceding image, we can see how we can roughly divide the image into interesting and uninteresting parts. We need to make sure that our algorithm detects these uninteresting parts and removes them. Let's consider the ducks image and the constraints we have to work with. We need to keep the height constant. This means that we need to find vertical seams in the image and remove them. These seams start at the top and end at the bottom (or vice versa). If we were dealing with vertical resizing, then the seams would start on the left-hand side and end on the right. A vertical seam is just a bunch of connected pixels starting at the top row and ending at the last row in the image.

# How do we define "interesting"?

Before we start computing the seams, we need to find out what metric we will be using to compute these seams. We need a way to assign "importance" to each pixel so that we can find out the paths that are least important. In computer vision terminology, we say that we need to assign an energy value to each pixel so that we can find the path of minimum energy. Coming up with a good way to assign the energy value is very important because it will affect the quality of the output.

One of the metrics that we can use is the value of the derivative at each point. This is a good indicator of the level of activity in that neighborhood. If there is some activity, then the pixel values will change rapidly. Hence the value of the derivative at that point would be high. On the other hand, if the region were plain and uninteresting, then the pixel values wouldn't change as rapidly. So, the value of the derivative at that point in the grayscale image would be low.

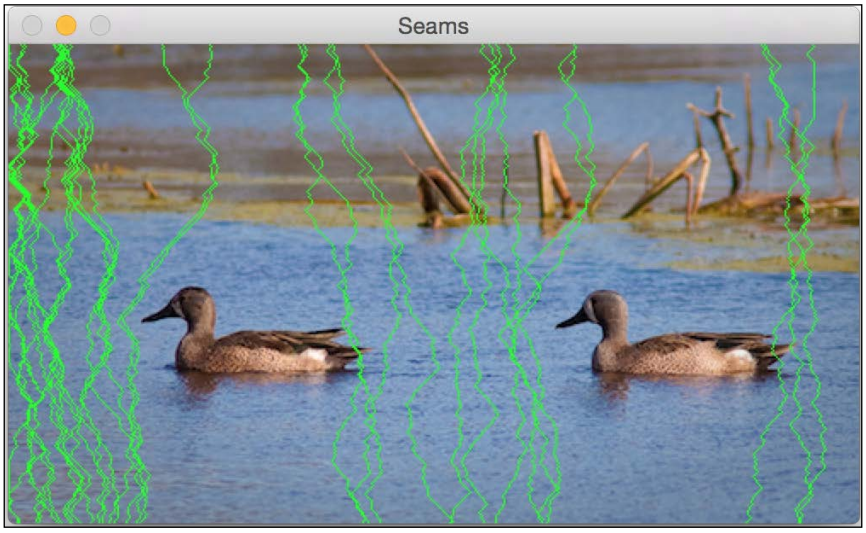
For each pixel location, we compute the energy by summing up the X and Y derivatives at that point. We compute the derivatives by taking the difference between the current pixel and its neighbors. If you recall, we did something similar to this when we were doing edge detection using **Sobel Filter** in *Chapter 1*, *Detecting Edges and Applying Image Filters*. Once we compute these values, we store them in a matrix called the energy matrix.

# How do we compute the seams?

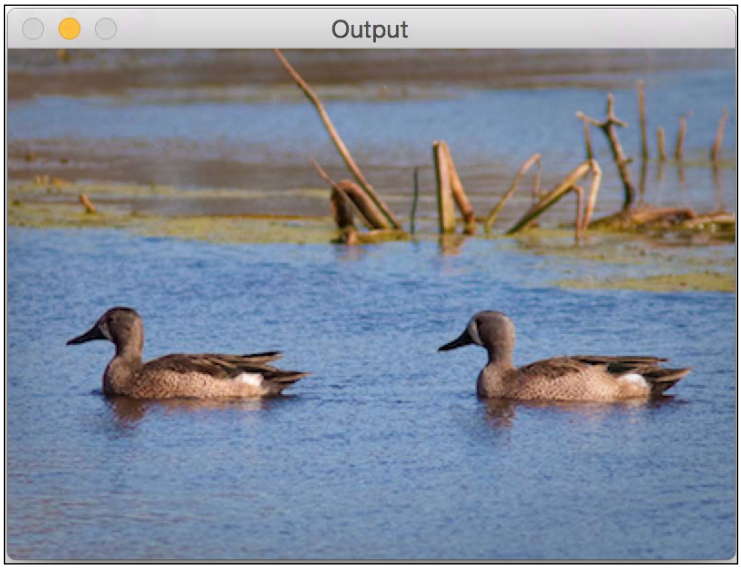
Now that we have the energy matrix, we are ready to compute the seams. We need to find the path through the image with the least energy. Computing all the possible paths is prohibitively expensive, so we need to find a smarter way to do this. This is where dynamic programming comes into the picture. In fact, seam carving is a direct application of dynamic programming. We need to start with each pixel in the first row and find our way to the last row. In order to find the path of least energy, we compute and store the best paths to each pixel in a table. Once we've construct this table, the path to a particular pixel can be found by backtracking through the rows in that table.

For each pixel in the current row, we calculate the energy of three possible pixel locations in the next row that we can move to, that is, bottom left, bottom, and bottom right. We keep repeating this process until we reach the bottom. Once we reach the bottom, we take the one with the least cumulative value and backtrack our way to the top. This will give us the path of least energy. Every time we remove a seam, the width of the image decreases by `1`. So we need to keep removing these seams until we arrive at the required image size.

Let's consider our ducks image again. If you compute the first 30 seams, it will look something like this:



These green lines indicate the paths of least importance. As we can see here, they carefully go around the ducks to make sure that the interesting regions are not touched. In the upper half of the image, the seams go around the twigs so that the quality is preserved. Technically speaking, the twigs are also "interesting". If you continue and remove the first 100 seams, it will look something like this:




Now, compare this with the naively resized image. Doesn't it look much better? The ducks look nice in this image.

Let's take a look at the code and see how to do it:

```
import sys

import cv2
import numpy as np

# Draw vertical seam on top of the image
def overlay_vertical_seam(img, seam):
    img_seam_overlay = np.copy(img) x

    # Extract the list of points from the seam
    x_coords, y_coords = np.transpose([(i,int(j)) for i,j in
enumerate(seam)])

    # Draw a green line on the image using the list of points
    img_seam_overlay[x_coords, y_coords] = (0,255,0)
```

```
    return img_seam_overlay

# Compute the energy matrix from the input image
def compute_energy_matrix(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Compute X derivative of the image
    sobel_x = cv2.Sobel(gray,cv2.CV_64F,1,0,ksize=3)

    # Compute Y derivative of the image
    sobel_y = cv2.Sobel(gray,cv2.CV_64F,0,1,ksize=3)

    abs_sobel_x = cv2.convertScaleAbs(sobel_x)
    abs_sobel_y = cv2.convertScaleAbs(sobel_y)

    # Return weighted summation of the two images i.e. 0.5*X +
0.5*Y
    return cv2.addWeighted(abs_sobel_x, 0.5, abs_sobel_y, 0.5, 0)

# Find vertical seam in the input image
def find_vertical_seam(img, energy):
    rows, cols = img.shape[:2]

    # Initialize the seam vector with 0 for each element
    seam = np.zeros(img.shape[0])

    # Initialize distance and edge matrices
    dist_to = np.zeros(img.shape[:2]) + sys.maxint
    dist_to[0,:] = np.zeros(img.shape[1])
    edge_to = np.zeros(img.shape[:2])

    # Dynamic programming; iterate using double loop and compute
the paths efficiently
    for row in xrange(rows-1):
        for col in xrange(cols):
            if col != 0:
                if dist_to[row+1, col-1] > dist_to[row, col] +
energy[row+1, col-1]:
                    dist_to[row+1, col-1] = dist_to[row, col] +
energy[row+1, col-1]
                    edge_to[row+1, col-1] = 1

            if dist_to[row+1, col] > dist_to[row, col] +
energy[row+1, col]:
```

```
                dist_to[row+1, col] = dist_to[row, col] +
energy[row+1, col]
                edge_to[row+1, col] = 0

            if col != cols-1:
                if dist_to[row+1, col+1] > dist_to[row, col] +
energy[row+1, col+1]:
                    dist_to[row+1, col+1] = dist_to[row, col] +
energy[row+1, col+1]
                    edge_to[row+1, col+1] = -1

    # Retracing the path
    seam[rows-1] = np.argmin(dist_to[rows-1, :])
    for i in (x for x in reversed(xrange(rows)) if x > 0):
        seam[i-1] = seam[i] + edge_to[i, int(seam[i])]

    return seam

# Remove the input vertical seam from the image
def remove_vertical_seam(img, seam):
    rows, cols = img.shape[:2]

    # To delete a point, move every point after it one step
towards the left
    for row in xrange(rows):
        for col in xrange(int(seam[row]), cols-1):
            img[row, col] = img[row, col+1]

    # Discard the last column to create the final output image
    img = img[:, 0:cols-1]
    return img

if __name__=='__main__':
    # Make sure the size of the input image is reasonable.
    # Large images take a lot of time to be processed.
    # Recommended size is 640x480.
    img_input = cv2.imread(sys.argv[1])

    # Use a small number to get started. Once you get an
    # idea of the processing time, you can use a bigger number.
    # To get started, you can set it to 20.
    num_seams = int(sys.argv[2])
```

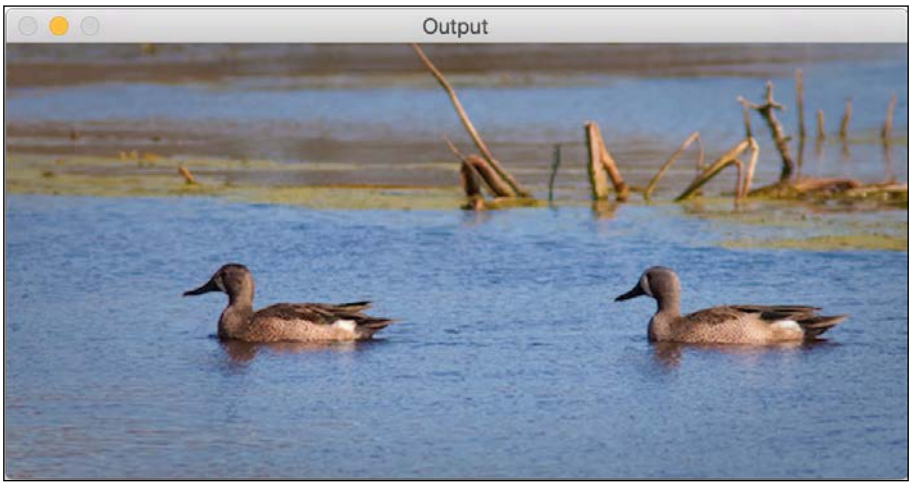
```
    img = np.copy(img_input)
    img_overlay_seam = np.copy(img_input)
    energy = compute_energy_matrix(img)

    for i in xrange(num_seams):
        seam = find_vertical_seam(img, energy)
        img_overlay_seam = overlay_vertical_seam(img_overlay_seam,
seam)
        img = remove_vertical_seam(img, seam)
        energy = compute_energy_matrix(img)
        print 'Number of seams removed =', i+1

    cv2.imshow('Input', img_input)
    cv2.imshow('Seams', img_overlay_seam)
    cv2.imshow('Output', img)
    cv2.waitKey()
```

# Can we expand an image?

We know that we can use seam carving to reduce the width of an image without deteriorating the interesting regions. So naturally, we need to ask ourselves if we can expand an image without deteriorating the interesting regions? As it turns out, we can do it using the same logic. When we compute the seams, we just need to add an extra column instead of deleting it.

If you expand the ducks image naively, it will look something like this:

If you do it in a smarter way, that is, by using seam carving, it will look something like this:




As you can see here, the width of the image has increased and the ducks don't look stretched. Following is the code to do it:

```
import sys

import cv2
import numpy as np

# Compute the energy matrix from the input image
def compute_energy_matrix(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
    sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
    abs_sobel_x = cv2.convertScaleAbs(sobel_x)
    abs_sobel_y = cv2.convertScaleAbs(sobel_y)
    return cv2.addWeighted(abs_sobel_x, 0.5, abs_sobel_y, 0.5, 0)

# Find the vertical seam
def find_vertical_seam(img, energy):
    rows, cols = img.shape[:2]

    # Initialize the seam vector with 0 for each element
    seam = np.zeros(img.shape[0])
```

```
    # Initialize distance and edge matrices
    dist_to = np.zeros(img.shape[:2]) + sys.maxint
    dist_to[0,:] = np.zeros(img.shape[1])
    edge_to = np.zeros(img.shape[:2])

    # Dynamic programming; iterate using double loop and compute
    #the paths efficiently
    for row in xrange(rows-1):
        for col in xrange(cols):
            if col != 0:
                if dist_to[row+1, col-1] > dist_to[row, col] +
                energy[row+1, col-1]:
                    dist_to[row+1, col-1] = dist_to[row, col] +
                    energy[row+1, col-1]
                    edge_to[row+1, col-1] = 1

            if dist_to[row+1, col] > dist_to[row, col] +
            energy[row+1, col]:
                dist_to[row+1, col] = dist_to[row, col] +
                energy[row+1, col]
                edge_to[row+1, col] = 0

            if col != cols-1:
                if dist_to[row+1, col+1] > dist_to[row, col] +
                energy[row+1, col+1]:
                    dist_to[row+1, col+1] = dist_to[row, col] +
                    energy[row+1, col+1]
                    edge_to[row+1, col+1] = -1

    # Retracing the path
    seam[rows-1] = np.argmin(dist_to[rows-1, :])
    for i in (x for x in reversed(xrange(rows)) if x > 0):
        seam[i-1] = seam[i] + edge_to[i, int(seam[i])]

    return seam

# Add a vertical seam to the image
def add_vertical_seam(img, seam, num_iter):
    seam = seam + num_iter
    rows, cols = img.shape[:2]
    zero_col_mat = np.zeros((rows,1,3), dtype=np.uint8)
    img_extended = np.hstack((img, zero_col_mat))
```

```
        for row in xrange(rows):
            for col in xrange(cols, int(seam[row]), -1):
                img_extended[row, col] = img[row, col-1]

            # To insert a value between two columns, take the average
            # value of the neighbors. It looks smooth this way and we
            # can avoid unwanted artifacts.
            for i in range(3):
                v1 = img_extended[row, int(seam[row])-1, i]
                v2 = img_extended[row, int(seam[row])+1, i]
                img_extended[row, int(seam[row]), i] =
                (int(v1)+int(v2))/2

        return img_extended

    # Remove vertical seam from the image
    def remove_vertical_seam(img, seam):
        rows, cols = img.shape[:2]
        for row in xrange(rows):
            for col in xrange(int(seam[row]), cols-1):
                img[row, col] = img[row, col+1]

        img = img[:, 0:cols-1]
        return img

    if __name__=='__main__':
        img_input = cv2.imread(sys.argv[1])
        num_seams = int(sys.argv[2])
        img = np.copy(img_input)
        img_output = np.copy(img_input)
        energy = compute_energy_matrix(img)

        for i in xrange(num_seams):
            seam = find_vertical_seam(img, energy)
            img = remove_vertical_seam(img, seam)
            img_output = add_vertical_seam(img_output, seam, i)
            energy = compute_energy_matrix(img)
            print 'Number of seams added =', i+1

        cv2.imshow('Input', img_input)
        cv2.imshow('Output', img_output)
        cv2.waitKey()
```

We added an extra function, `add_vertical_seam`, in this code. We use it to add vertical seams so that the image looks natural.

# Can we remove an object completely?

This is perhaps the most interesting application of seam carving. We can make an object completely disappear from an image. Let's consider the following image:

Let's select the region of interest:

After you remove the chair on the right, it will look something like this:


It's as if the chair never existed! Before we look at the code, it's important to know that this takes a while to run. So, just wait for a couple of minutes to get an idea of the processing time. You can adjust the input image size accordingly! Let's take a look at the code:

```
import sys

import cv2
import numpy as np

# Draw rectangle on top of the input image
def draw_rectangle(event, x, y, flags, params):
    global x_init, y_init, drawing, top_left_pt, bottom_right_pt,
    img_orig

    # Detecting a mouse click
    if event == cv2.EVENT_LBUTTONDOWN:
        drawing = True
        x_init, y_init = x, y

    # Detecting mouse movement
    elif event == cv2.EVENT_MOUSEMOVE:
        if drawing:
            top_left_pt, bottom_right_pt = (x_init,y_init), (x,y)
            img[y_init:y, x_init:x] = 255 - img_orig[y_init:y,
            x_init:x]
```

```
            cv2.rectangle(img, top_left_pt, bottom_right_pt,
            (0,255,0), 2)

    # Detecting the mouse button up event
    elif event == cv2.EVENT_LBUTTONUP:
        drawing = False
        top_left_pt, bottom_right_pt = (x_init,y_init), (x,y)

        # Create the "negative" film effect for the selected
        # region
        img[y_init:y, x_init:x] = 255 - img[y_init:y, x_init:x]

        # Draw rectangle around the selected region
        cv2.rectangle(img, top_left_pt, bottom_right_pt,
        (0,255,0), 2)
        rect_final = (x_init, y_init, x-x_init, y-y_init)

        # Remove the object in the selected region
        remove_object(img_orig, rect_final)

# Computing the energy matrix using modified algorithm
def compute_energy_matrix_modified(img, rect_roi):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Compute the X derivative
    sobel_x = cv2.Sobel(gray,cv2.CV_64F,1,0,ksize=3)

    # Compute the Y derivative
    sobel_y = cv2.Sobel(gray,cv2.CV_64F,0,1,ksize=3)
    abs_sobel_x = cv2.convertScaleAbs(sobel_x)
    abs_sobel_y = cv2.convertScaleAbs(sobel_y)

    # Compute weighted summation i.e. 0.5*X + 0.5*Y
    energy_matrix = cv2.addWeighted(abs_sobel_x, 0.5, abs_sobel_y,
    0.5, 0)
    x,y,w,h = rect_roi

    # We want the seams to pass through this region, so make sure the
energy values in this region are set to 0
    energy_matrix[y:y+h, x:x+w] = 0

    return energy_matrix
```

```
# Compute energy matrix
def compute_energy_matrix(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Compute X derivative
    sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)

    # Compute Y derivative
    sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
    abs_sobel_x = cv2.convertScaleAbs(sobel_x)
    abs_sobel_y = cv2.convertScaleAbs(sobel_y)

    # Return weighted summation i.e. 0.5*X + 0.5*Y
    return cv2.addWeighted(abs_sobel_x, 0.5, abs_sobel_y, 0.5, 0)

# Find the vertical seam
def find_vertical_seam(img, energy):
    rows, cols = img.shape[:2]

    # Initialize the seam vector
    seam = np.zeros(img.shape[0])

    # Initialize the distance and edge matrices
    dist_to = np.zeros(img.shape[:2]) + sys.maxint
    dist_to[0,:] = np.zeros(img.shape[1])
    edge_to = np.zeros(img.shape[:2])

    # Dynamic programming; using double loop to compute the paths
    for row in xrange(rows-1):
        for col in xrange(cols):
            if col != 0:
                if dist_to[row+1, col-1] > dist_to[row, col] +
                energy[row+1, col-1]:
                    dist_to[row+1, col-1] = dist_to[row, col] +
                    energy[row+1, col-1]
                    edge_to[row+1, col-1] = 1

            if dist_to[row+1, col] > dist_to[row, col] +
            energy[row+1, col]:
                dist_to[row+1, col] = dist_to[row, col] +
                energy[row+1, col]
                edge_to[row+1, col] = 0
```

```
            if col != cols-1:
                if dist_to[row+1, col+1] > dist_to[row, col] +
                energy[row+1, col+1]:
                    dist_to[row+1, col+1] = dist_to[row, col] +
                    energy[row+1, col+1]
                    edge_to[row+1, col+1] = -1

    # Retracing the path
    seam[rows-1] = np.argmin(dist_to[rows-1, :])
    for i in (x for x in reversed(xrange(rows)) if x > 0):
        seam[i-1] = seam[i] + edge_to[i, int(seam[i])]

    return seam

# Add vertical seam to the input image
def add_vertical_seam(img, seam, num_iter):
    seam = seam + num_iter
    rows, cols = img.shape[:2]
    zero_col_mat = np.zeros((rows,1,3), dtype=np.uint8)
    img_extended = np.hstack((img, zero_col_mat))

    for row in xrange(rows):
        for col in xrange(cols, int(seam[row]), -1):
            img_extended[row, col] = img[row, col-1]

        # To insert a value between two columns, take the average
        # value of the neighbors. It looks smooth this way and we
        # can avoid unwanted artifacts.
        for i in range(3):
            v1 = img_extended[row, int(seam[row])-1, i]
            v2 = img_extended[row, int(seam[row])+1, i]
            img_extended[row, int(seam[row]), i] = (int(v1)+int(v2))/2

    return img_extended

# Remove vertical seam
def remove_vertical_seam(img, seam):
    rows, cols = img.shape[:2]
    for row in xrange(rows):
        for col in xrange(int(seam[row]), cols-1):
            img[row, col] = img[row, col+1]

    img = img[:, 0:cols-1]
    return img
```

```
# Remove the object from the input region of interest
def remove_object(img, rect_roi):
    num_seams = rect_roi[2] + 10
    energy = compute_energy_matrix_modified(img, rect_roi)

    # Start a loop and remove one seam at a time
    for i in xrange(num_seams):
        # Find the vertical seam that can be removed
        seam = find_vertical_seam(img, energy)

        # Remove that vertical seam
        img = remove_vertical_seam(img, seam)
        x,y,w,h = rect_roi

        # Compute energy matrix after removing the seam
        energy = compute_energy_matrix_modified(img, (x,y,w-i,h))
        print 'Number of seams removed =', i+1

    img_output = np.copy(img)

    # Fill up the region with surrounding values so that the size
    # of the image remains unchanged
    for i in xrange(num_seams):
        seam = find_vertical_seam(img, energy)
        img = remove_vertical_seam(img, seam)
        img_output = add_vertical_seam(img_output, seam, i)
        energy = compute_energy_matrix(img)
        print 'Number of seams added =', i+1

    cv2.imshow('Input', img_input)
    cv2.imshow('Output', img_output)
    cv2.waitKey()

if __name__=='__main__':
    img_input = cv2.imread(sys.argv[1])

    drawing = False
    img = np.copy(img_input)
    img_orig = np.copy(img_input)

    cv2.namedWindow('Input')
    cv2.setMouseCallback('Input', draw_rectangle)
```

```
while True:
    cv2.imshow('Input', img)
    c = cv2.waitKey(10)
    if c == 27:
        break

cv2.destroyAllWindows()
```

# How did we do it?

The basic logic remains the same here. We are using seam carving to remove an object. Once we select the region of interest, we make all the seams pass through this region. We do this by manipulating the energy matrix after every iteration. We have added a new function called `compute_energy_matrix_modified` to achieve this. Once we compute the energy matrix, we assign a value of 0 to this region of interest. This way, we force all the seams to pass through this area. After we remove all the seams related to this region, we keep adding the seams until we expand the image to its original width.

# Summary

In this chapter, we learned about content-aware image resizing. We discussed how to quantify interesting and uninteresting regions in an image. We learned how to compute seams in an image and how to use dynamic programming to do it efficiently. We discussed how to use seam carving to reduce the width of an image, and how we can use the same logic to expand an image. We also learned how to remove an object from an image completely.

In the next chapter, we are going to discuss how to do shape analysis and image segmentation. We will see how to use those principles to find the exact boundaries of an object of interest in the image.